

Stanford CS329A 课程补编
把《Advanced LLM Agents》推进到《Self-Improving AI
Agents》

Supplement workspace for CS294/194-280 Spring 2025 textbook

2026-04-16

目录

第一章 Stanford CS329A 课程补编：把 “Advanced LLM Agents” 推进到 “Self-Improving AI Agents”

1.1 补编定位与证据边界

本补编服务于现有的 Berkeley 《CS294/194-280: Advanced Large Language Model Agents, Spring 2025》中文教材。它不是一门新书的完整重建，而是一个课程级扩展章：用 Stanford《CS329A: Self-Improving AI Agents, Autumn 2025》的官方课程页与官方 schedule，回答一个更强的问题：如果把 Spring 2025 Berkeley 课程看成 “advanced LLM agents” 的系统教材，那么 Stanford CS329A 把这条线进一步推进到了哪里？

这里必须先说清楚证据边界：

- 本补编的 canonical source 只有两个：<https://cs329a.stanford.edu/> 与 <https://cs329a.stanford.edu/#schedule>。
- 我没有在 Stanford 官方课程站点上找到官方公开视频页。
- 我没有在 Stanford 官方课程站点上找到官方 slide 页面或 lecture slide 索引。
- 因此，本补编不是逐讲视频重建，也不是 slide 逐页讲义，而是基于官方课程概览、官方 schedule 结构、官方 reading 标题和课程组织方式做出的教材化扩展。
- 存在一个公开 YouTube playlist，但它更适合被描述为 instructor-affiliated，而不是 Stanford 课程官网明确公布的官方 source-of-truth，所以这里只做说明，不把它当作覆盖依据。

这意味着本补编有一个非常重要的方法论约束：凡是超出官方课程页和 reading 标题字面信息的解释，都应被理解为教材化推断或结构化解读，而不是对未公开 lecture 内容的伪装复原。

1.2 为什么 CS329A 是现有 Berkeley 教材的强补充

Berkeley Spring 2025 的主线，是把 advanced LLM agents 拆成几个核心子系统：推理时计算、学习推理、记忆与规划、coding agents、多模态 agents、formal mathematics、abstraction/discovery，以及安全与权限边界。

Stanford CS329A 的官方课程概览则把这些对象重新组织成一个更强的系统目标：self-improvement。课程页没有只说 agent 很强，而是明确列出一条自我改进链路：constitutional AI、verifiers、scaling

test-time compute、combining search with LLMs、train-time scaling with RL、tool use、code、memory、multimodal interaction、multi-step reasoning and planning、evaluation frameworks、coding agents、research assistants in STEM，以及 robotics。

如果把这条链路教材化，可以写成下面这个抽象循环：

$$\pi_{t+1} = \mathcal{U}(\pi_t, \mathcal{T}, \mathcal{V}, \mathcal{M}, \mathcal{S}, \mathcal{D})$$

这里的写法不是 Stanford 官网原文，而是对官方课程概览的教材化抽象：

- π_t ：第 t 轮 agent 系统。
- $\mathcal{T}$ ：tools 与 code execution 反馈。
- $\mathcal{V}$ ：verifiers 与外部检查器。
- $\mathcal{M}$ ：memory 与长期上下文结构。
- $\mathcal{S}$ ：search、planning 与 test-time compute。
- $\mathcal{D}$ ：训练数据、交互轨迹与评测反馈。
- $\mathcal{U}$ ：把这些反馈汇总成系统更新的机制。

Berkeley Spring 2025 更像是在解释这些部件各自如何工作；Stanford Autumn 2025 则更像是在问：怎样把这些部件串成一个持续改进的 agent loop。这正是它对现有教材的第一个关键补强。

1.3 第一条扩展主线：从“推理技巧”升级到“自我改进回路”

1.3.1 Test-time compute 不再只是推理技巧，而是自我改进的前半段

Stanford schedule 把 2025 年 9 月 26 日直接设成“Test-time Compute Scaling”，并配上 Large Language Monkeys、Archon、Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters，以及 How Do Large Language Monkeys Get Their Power (Laws)? 这些 reading。

只看标题就足够说明课程意图：这不是把 repeated sampling 或 search 当作零散 trick，而是把它们看成一个可被系统地扩展、比较、甚至形成 scaling law 的能力来源。Berkeley 的第 1 讲已经讲了 inference-time reasoning techniques，但 Stanford 在课程结构上又往前走了一步：它把 test-time compute 直接放进 self-improving agent 的总框架中。这意味着推理时计算不只是为了在一次任务上答对，而是为了给后续的 selection、verification、feedback learning 和 system redesign 提供更高质量的候选轨迹。

1.3.2 Robust Verification 把“验证器”提前变成中心对象

2025 年 9 月 29 日的主题是“Robust Verification”。相应 reading 包括 Shrinking the Generation-Verification Gap with Weak Verifiers、Training Verifiers to Solve Math Word Problems、Let’s Verify Step by Step，以及 Math-Shepherd。

这和 Berkeley Spring 2025 的一个差异非常关键。Berkeley 的 verifier 主要出现在推理、formal mathematics 和 theorem proving 这些部分；而 Stanford 在课表编排上更像是在说：没有 verifier，就谈不上 reliable self-improvement。

1.4 第二条扩展主线：从 DISCOVERY 章节走向 OPEN-ENDED RESEARCH AUTOMATION7

这会直接改变教材中的逻辑顺序。原书里，verifier 往往被读者理解为某些高难数学或形式化任务的附加组件；但从 CS329A 的编排看，verifier 更像是所有 agent 改进循环里的通用裁判。只要 agent 要从自己的候选行为中学习，generation-verification gap 就会成为首要问题。

1.3.3 Tool/code feedback 和 planning 被合并进同一个闭环

2025 年 10 月 3 日是 “Learning from feedback with tools/code”，10 月 6 日是 “Multi-step Reasoning/Planning”。这两次课连在一起看，比单独看更重要。

官方 reading 标题给出的信号是：ReAct 说明 reasoning 与 acting 要耦合；RLEF 说明 code execution feedback 可以成为强化学习信号；Constitutional AI 说明 feedback 不一定来自人，也可以来自规则化的 AI feedback；而 SWiRL、Language Agent Tree Search、SPRINT、ADaPT 与 Wider or Deeper? 则把多步推理、树搜索、分解规划、并行执行都拉进来。

因此，在 Stanford 的课程设计里，tool use 不是“会不会调 API”这种孤立技能，planning 也不是“让模型多想一步”的装饰。二者被放在一起，是因为有工具的 environment 才会产生足够强的反馈，而多步规划又决定 agent 如何消费这些反馈。

把它写成一个教材化伪代码，更接近 CS329A 的课程精神：

```
Initialize agent policy and tool interface
Generate multi-step plans under a limited compute budget
Execute tool/code actions and collect environment feedback
Use verifiers or rule-based critics to filter candidate trajectories
Store useful traces, failures, and reusable subplans
Update the agent or search policy for the next round
```

这段伪代码不是官网原文，但它忠实反映了官方 overview 与 schedule 的组合含义：planning、tool use、feedback learning、memory 与 verification 被组织成一个统一的 self-improvement workflow。

1.3.4 Train-time scaling/Scaling RL 补上自我改进的后半段

2025 年 10 月 10 日的主题是 “Train Time Scaling/Scaling RL”，reading 包括 STaR、DeepSeek-Math 和 DAPO。

Berkeley Spring 2025 里，读者已经见过 DPO/PPO、reasoning post-training 和 formal RL 在特定 lecture 中的作用。Stanford 这里的增量不是“又多几篇 RL 论文”，而是把 train-time scaling 明确写成自我改进回路的另一半：前半段是 test-time compute、search、planning、verification，后半段则是如何把这些交互轨迹回流为训练信号。

1.4 第二条扩展主线：从 discovery 章节走向 open-ended research automation

1.4.1 Open-Ended Evolution 把发现问题本身推到前台

2025 年 10 月 13 日的主题是 “Open-Ended Evolution of Self-Improving Agents”。官方 reading 标题包括 Automated design of agentic systems、The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery，以及 AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms。

Berkeley Spring 2025 的第 11 讲已经讨论了 abstraction 和 discovery, 但 Stanford 在课程命名上把“open-ended evolution”直接放进主干。这个差别很大: Berkeley 更像在问 agent 如何帮助 theorem proving、abstraction 和 scientific discovery; Stanford 更像在问 agent 能否持续地设计新的 agent system、新的 research workflow, 甚至新的 algorithmic artifact。

1.4.2 Search & Deep Research Agents 把 coding 和 research synthesis 连到一起

2025 年 10 月 17 日的主题是“Self improvement with Search & Deep Research Agents”, reading 包括 Competition-Level Code Generation with AlphaCode、AlphaCode 2 Technical Report, 以及 Search-01: Agentic Search-Enhanced Large Reasoning Models。

单看标题, 就可以看出 Stanford 想把两件原本常被分开的事情放在一起: 代码与程序搜索, 以及 research-style 搜索与综合。这对现有 Berkeley 教材的意义是: 原书已经有 coding agents、formal mathematics、abstraction/discovery 三条线, 但 Stanford 的补充会迫使我们把它们读成一条连续谱: 代码生成是可执行搜索, theorem proving 是 verifier-backed 搜索, deep research 则是资料、假设与证据之间的开放式搜索。

1.5 第三条扩展主线: 把 agent infrastructure 讲得更像系统工程

1.5.1 从 chatbots 到 agents: post-training 被重新定义

2025 年 10 月 20 日的 guest lecture 题目是“Evolution of Post-training from Chatbots to Agents”。虽然官方 schedule 没有给 reading, 但这个标题本身就很值得写进补编。它说明 Stanford 把 post-training 看成 agent emergence 的制度性条件, 而不只是对话模型礼貌化或 alignment 的尾声。

因为官方站点没有公开 lecture 细节, 这里不能虚构具体方法; 但课程题目已经足够说明其教学方向: agentic behavior 不是只靠 inference-time prompting 长出来的, post-training 本身也在发生从 chatbot 到 agent 的范式迁移。

1.5.2 Agentic Frameworks for Software Engineering 让 coding agent 更接近“生产系统”

2025 年 11 月 3 日的主题是“Agentic Frameworks for Software Engineering”, reading 包括 CodeMonkeys、KernelBench, 以及 Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces。

这组标题直接把 Berkeley 第 5 讲的 coding agent 讨论往前推了一层。Spring 2025 里, 我们已经看到 coding agents 与 vulnerability detection; 但 Stanford 这里强调的是: software engineering 是 test-time compute 的落地场景, benchmark 不该只看 pass@k, 还要看性能、系统接口和并行程序优化, agent 不是单一模型, 而是 model + interface + runtime + evaluator 的整体。

1.5.3 Memory 被提升为长期系统能力, 而不是局部增强技巧

2025 年 11 月 7 日的 guest lecture 是“Augmenting Agents with Memory”, reading 包括 Cartridges、MemGPT, 以及 CacheBlend。

这与 Berkeley 第 3 讲形成直接呼应。Spring 2025 的教材里，memory 已经出现，但 Stanford 的课表说明 memory 现在更像一个完整基础设施层：它关系到长期任务分解、上下文复用与缓存，也关系到 agent 是否能把过去的交互沉淀成未来的能力。

1.6 第四条扩展主线：把 evaluation 放到 agent loop 的中轴

1.6.1 late-term reasoning guest lectures 的信号

2025 年 11 月 10 日的 guest lecture 是 “LLM Reasoning”，11 月 14 日是 “Towards AI Super-human Reasoning: AlphaProof, AlphaGeometry & Gemini IMO Gold Medal”。

由于官方站点没有给出 reading，这里不能展开成独立技术综述；但它们在课程后半段的出现，已经能说明两件事：第一，Stanford 没有把 reasoning 看成开头讲完就结束的话题，而是把它反复拉回课程主干；第二，reasoning 的标尺被继续抬高到了 formal mathematics、geometry 和竞赛级表现附近。

1.6.2 Agentic Evaluations & Long-Horizon Tasks 改写了“评测”的位置

2025 年 11 月 17 日的主题是 “Agentic Evaluations & Long-Horizon Tasks”，reading 包括 Measuring AI Ability to Complete Long Tasks、GDPVal，以及 DeepScholar-Bench。

这一组标题的意义非常大。Berkeley Spring 2025 当然也谈 evaluation 与 safety，但 Stanford 这里的官方安排表明：评测不再只是最后验收模型分数，评测定义了 agent 究竟在什么环境里被算作“改进了”，而长时程任务、经济价值任务、研究综合任务，都成为新的 primary target。

1.6.3 项目制结构本身也是 evaluation philosophy 的一部分

Stanford 官方页还清楚列出了 Homework 1、2、3, Project Proposal, Midterm presentation + Report, Final project, 以及 Poster。这说明课程并不把评测局限于论文阅读或考试，而是把“提出问题、搭系统、做中期汇报、跑最终项目”本身当作学习闭环的一部分。

1.7 第五条扩展主线：从多模态 agent 走向 autonomy 与 robotics

2025 年 11 月 21 日的 guest lecture 是 “Building Agentic Systems for Autonomy: Lessons & Open questions”，12 月 1 日是 “Multimodal AI Agents in Robotics”。

官方站点同样没有给出 lecture 细节，所以这里必须保持克制。但课程结构已经非常明确：Stanford 把 autonomy 与 robotics 放在课程收尾，说明它们被看成对前面所有能力的综合检验。真正进入 autonomy 或 robotics，agent 需要同时处理 perception、action、memory、planning、environment feedback、robustness，以及 long-horizon correction。这正好把 Berkeley 第 6、7 讲的多模态与 perception-to-action 主题，推进到“实际 autonomy systems”这一层。

1.8 如何用这门课来重构现有 Berkeley 教材

现有 Berkeley 教材部分	Stanford CS329A 的增量	应如何改写原书理解
推理时计算、学习推理	Test-time Compute Scaling、Robust Verification、Train Time Scaling/Scaling RL	把 reasoning 写成一个从 test-time 到 train-time 的 self-improvement loop。
Reasoning / Memory / Planning	Multi-step Reasoning/Planning、Augmenting Agents with Memory	把 planning 和 memory 视为长期自我改进的运行时基础设施。
Coding Agents	Learning from feedback with tools/code、Agentic Frameworks for Software Engineering、AlphaCode 2	从“会写代码”升级到“能在系统接口与执行反馈中持续优化”。
Abstraction / Discovery	Open-Ended Evolution of Self-Improving Agents、AI Scientist、AlphaEvolve	把 discovery 写成 open-ended research automation，而不仅是单点 scientific insight。
Formal math / theorem proving	Robust Verification、AlphaProof 等 late-term reasoning 信号	把 verifier 和 formal reasoning 提前，作为通用 agent 改进机制。
Multimodal agents	Multimodal AI Agents in Robotics、Autonomy	把多模态从 perception demo 推向 embodied long-horizon autonomy。
安全与评测	Agentic Evaluations & Long-Horizon Tasks、项目制课程结构	把 evaluation 变成 agent loop 的中轴，而不是章节尾声。

1.9 本补编的使用方式

如果你已经读完 Berkeley Spring 2025 这本书，建议用下面的顺序吸收这个 Stanford 补编：

1. 先重读 Berkeley 的第 1-4 讲，把 reasoning、planning、post-training 重新放进 self-improvement 框架。
2. 再回看 Berkeley 的 coding agents、formal mathematics 和 abstraction/discovery 几章，把它们读成 research automation 与 open-ended system design 的连续谱。
3. 最后把多模态 agent、安全与评测章节重新阅读，重点关注 long-horizon tasks、economic value、research synthesis 和 robotics 这几个新的评价维度。

1.10 总结

如果说 Berkeley Spring 2025 教材回答的是“高级 LLM agent 由哪些关键部件构成”，那么 Stanford CS329A 进一步追问的是：当这些部件被放进长期反馈回路后，agent 如何持续改进自己？

这门课最重要的补充，不是单个新名词，而是一个课程级重心变化：reasoning 不再只是求解，verifier 不再只是附加检查器，tools 与 code 不再只是插件，memory 不再只是辅助缓存，evaluation

不再只是结果统计。它们一起变成了 self-improvement system 的组成部分。

从教材工程的角度看，这意味着现有 Berkeley 讲义项目后续如果继续扩展，最值得优先补写的，不是“再多几个案例”，而是把以下五条主线更清楚地前置出来：

1. self-improvement loop
2. verifier-centric learning
3. coding and research automation
4. long-horizon evaluation
5. autonomy and robotics as integrated end-to-end tests

这正是 Stanford CS329A 对整本书最有价值的增量。